Lab 3: HTTP Protocol Analysis with CML and Wireshark

1. Background

The Hypertext Transfer Protocol (HTTP) is the foundation of data communication for the World Wide Web. HTTP functions as a request-response protocol in the client-server model. A client (such as a web browser or command-line utility like `curl` or `wget`) sends an HTTP request message to the server, and the server replies with an HTTP response message containing status information and the requested content.

In this lab, you will deploy a local web client and server within a simulated environment using Cisco Modeling Labs (CML). You will analyze the format of HTTP messages, explore conditional GET requests used for caching, examine how large documents are segmented by the transport layer, observe the retrieval of documents with embedded objects, and inspect the mechanics of HTTP Basic Authentication.

---

2. Objective / Purpose

The objective of this lab is to build upon your CML and Wireshark skills by constructing a client-server network topology, generating various types of HTTP traffic, and capturing the resulting packets for analysis.

Specifically, you will:

- Deploy a **Firefox** client and an **Nginx** web server in CML.
- Configure automatic startup scripts for both nodes to initialize the web files and server configuration.
- Generate and capture HTTP traffic for basic GET requests, conditional GET requests, segmented transfers, embedded objects, and basic authentication.
- Analyze the captured HTTP headers and TCP structures in Wireshark to answer key conceptual questions.

---

3. CML Topology and Configuration Procedure

```
graph LR
    H1["Firefox Client<br>OS: Docker VNC Desktop<br>eth0: 192.168.1.1/24"] <--> |"eth0 <-->
eth0<br>[Packet Capture Link]"| H2["Nginx Server<br>OS: Docker Container<br>eth0:
192.168.1.2/24<br>Services: Nginx (Port 80)"]

    classDef client fill:#d4f1f9,stroke:#007b8b,stroke-width:2px;
    classDef server fill:#e2f0d9,stroke:#385723,stroke-width:2px;

    class H1 client;
    class H2 server;
```

Step 1: Deploying the Nodes

1. Log in to the CML UI and open/create your lab. (If building on top of Lab 1, you can add to the existing topology or create a new lab).
2. Drag one **Firefox** node (rename it `<initials>-Firefox`) and one **Nginx** node (rename it `<initials>-Nginx`) onto the workspace.
3. Create a link connecting the two nodes (e.g., `eth0` on the Firefox Client to `eth0` on the Nginx Server).

Step 2: Configuring the Client and Server

You will use CML's **Edit Config** tab to inject configuration scripts that execute automatically when the nodes boot.

1. Firefox Client Configuration

- Click on the Firefox client node in the workspace.
- In the bottom pane, navigate to the **Edit Config** tab.
- Enter the following shell script to configure its static IP address, start the interface, and signal CML that the boot process is complete:

```
#!/bin/sh
echo "=== Starting Firefox CML Node Boot Script ===" >/dev/console

# Configure network interface eth0
echo "Configuring eth0 static IP..." >/dev/console
ip addr add 192.168.1.1/24 dev eth0
ip link set eth0 up
ip addr show dev eth0 >/dev/console

# Signal boot completion to CML
echo "READY" >/dev/console
exit 0
```

2. Nginx Server Configuration

- Click on the Nginx server node in the workspace.
- Navigate to the **Edit Config** tab.
- Enter the following shell script to configure its static IP address, create the test HTML files, configure the basic auth server block, test the configuration, reload Nginx, and print diagnostic information to the CML console:

```
#!/bin/bash
echo "=== Starting Nginx CML Node Boot Script ===" >/dev/console

# Configure network interface eth0
echo "Configuring eth0 static IP..." >/dev/console
ip addr add 192.168.1.2/24 dev eth0
ip link set eth0 up
ip addr show dev eth0 >/dev/console

# Wait for network interface initialization
sleep 1

# Create directories for web documents
echo "Creating web document directories..." >/dev/console
mkdir -p /usr/share/nginx/html/protected

# 1. Create a basic HTML file (index.html)
echo "Creating index.html..." >/dev/console
cat << 'EOF' > /usr/share/nginx/html/index.html
<html>
<body>
<h1>Welcome to CS457 Lab 3</h1>
```

```
<p>This is a simple HTML document with no embedded objects.</p>
</body>
</html>
EOF

# 2. Create a long document (long.html, ~10KB) to trigger TCP segmentation
echo "Creating long.html (~10KB)..." >/dev/console
dd if=/dev/urandom of=/usr/share/nginx/html/long.html bs=1024 count=10 >/dev/console 2>&1

# 3. Create HTML document with embedded object (embedded.html)
echo "Creating embedded.html & logo.png..." >/dev/console
cat << 'EOF' > /usr/share/nginx/html/embedded.html
<html>
<body>
<h1>Embedded Image Test</h1>
<img src="logo.png">
</body>
</html>
EOF

# Create dummy image file (~1KB)
dd if=/dev/zero of=/usr/share/nginx/html/logo.png bs=1024 count=1 >/dev/console 2>&1

# 4. Create protected folder content
echo "Creating protected directory and files..." >/dev/console
cat << 'EOF' > /usr/share/nginx/html/protected/index.html
<html>
<body>
<h1>Protected Content</h1>
<p>If you see this, authentication was successful!</p>
</body>
</html>
EOF

# 5. Set up Basic Authentication user file (cisco:password123)
echo "Configuring basic authentication credentials..." >/dev/console
echo 'cisco:$apr1$y8z7Z7u3$L0tH3V9P/XvD01Nl3R6c2.' > /etc/nginx/.htpasswd

# Write custom Nginx configuration block
echo "Writing default.conf server configuration..." >/dev/console
cat << 'EOF' > /etc/nginx/conf.d/default.conf
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
    }

    location /protected {
        root /usr/share/nginx/html;
```

```
        auth_basic "Restricted Area";
        auth_basic_user_file /etc/nginx/.htpasswd;
    }
}
EOF

# Verify Nginx configuration syntax
echo "Testing Nginx configuration..." >/dev/console
nginx -t >/dev/console 2>&1

# Reload Nginx to pick up the configuration if Nginx is already running
echo "Reloading Nginx server configuration..." >/dev/console
nginx -s reload >/dev/console 2>&1 || true

# List created files to console
echo "Listing generated HTML files:" >/dev/console
ls -R /usr/share/nginx/html >/dev/console

# Notify CML that the boot process has completed
echo "READY" >/dev/console
exit 0
```

Step 3: Starting the Nodes

1. Click the **Start Lab** button (play icon) at the top of the workspace to boot up both nodes.
2. Wait for the nodes to turn green, indicating they are fully active.

---

4. Lab Execution and Traffic Generation

Step 1: Starting the Packet Capture

1. Click on the link connecting your Firefox Client and Nginx Server.
2. In the bottom pane, navigate to the **Packet Capture** tab.
3. Click **Start** to begin capturing traffic on the link.

Step 2: Testing and Generating HTTP Traffic

1. Click on your Firefox Client node. In the bottom pane, click the **VNC** tab. (This will display a graphical interface showing the Linux desktop environment).
2. Open the **Firefox** browser from the desktop.
3. Open the **Firefox Developer Tools** by pressing **F12** (or clicking the menu icon -> More Tools -> Web Developer Tools). Select the **Network** tab.
4. Verify local connectivity by opening a terminal on the client desktop and pinging the server:

   ```
   ping -c 4 192.168.1.2
   ```

5. **Test 1: The Basic HTTP GET/Response**
   - In the Firefox address bar, enter: `http://192.168.1.2/index.html` and press **Enter**.
   - In the Developer Tools Network tab, click on the request for `index.html`.

- Under the **Headers** panel, view the Request and Response headers. Note the `Last-Modified` header printed in the server's response.

6. **Test 2: The HTTP Conditional GET**
   - Refresh the page in Firefox (press **F5** or the refresh icon).
   - Look at the new request in the Network tab. You should see a status of `304 Not Modified`.
   - Inspect the request headers: you will see an `If-Modified-Since` header containing the timestamp from the previous test. The server does not send the body content again because it detects the client already has it cached.
   - *Note: To force a full reload bypassing cache, hold Shift and press the reload button (should result in `200 OK`).*
7. **Test 3: Retrieving Long Documents**
   - In the Firefox address bar, enter: `http://192.168.1.2/long.html` and press **Enter**.
   - Since this file is large (~10KB), it exceeds the standard Ethernet MTU (1500 bytes) and will be segmented by TCP.
8. **Test 4: HTML Documents with Embedded Objects**
   - In the Firefox address bar, enter: `http://192.168.1.2/embedded.html` and press **Enter**.
   - Inspect the Network tab: you should see that Firefox automatically makes **two** separate GET requests—one for `embedded.html` and one for `logo.png`—as it parses the HTML and finds the image reference.
9. **Test 5: HTTP Basic Authentication**
   - In the Firefox address bar, enter: `http://192.168.1.2/protected/index.html` and press **Enter**.
   - A dialog popup should appear in the browser asking for username and password.
   - Enter `cisco` for the username and `password123` for the password.
   - Once loaded, inspect the request headers for the successful authentication request. You will see an `Authorization: Basic` header showing a base64 encoded credential string.

Step 3: Downloading the Capture

1. Go back to the **Packet Capture** tab for the link in CML and click **Stop**.
2. Click the **Download** button to save the `.pcap` capture file to your local machine.

---

5. Wireshark Analysis and Questions

Open your downloaded capture file in Wireshark and answer the following questions:

5.1 Basic GET/Response Interaction

1. Is your client running HTTP version 1.0, 1.1, or 2? What version of HTTP is the Nginx server running?
2. What languages (if any) does your client indicate in its headers that it can accept?
3. What is the IP address of the client? What is the IP address of the Nginx server?
4. What is the status code returned from the server to the client in response to the basic GET?
5. When was the `index.html` file last modified at the server?
6. How many bytes of content are being returned to the client (inspect the `Content-Length` header)?
7. By inspecting the raw data in the packet content window, do you see any headers within the data that are not displayed in the packet-listing window? If so, name one.

5.2 Conditional GET/Response Interaction

8. Inspect the contents of your first HTTP GET request from the client to the server. Do you see an `If-Modified-Since` header line in the HTTP GET?

9. Inspect the contents of the server response to that first GET. Did the server explicitly return the contents of the file? How can you tell?
10. Now inspect the contents of your second HTTP GET request (the conditional GET). Do you see an `If-Modified-Since:` header line? If so, what timestamp follows it?
11. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Explain.

5.3 Retrieving Long Documents

12. How many HTTP GET request messages did the client send? Which packet number in the trace contains the GET message for the long document?
13. Which packet number in the trace contains the HTTP status code and response associated with the response to this GET request?
14. What is the status code and phrase in the response?
15. How many data-containing TCP segments were needed to carry the single HTTP response containing the text of `long.html`?

5.4 Embedded Objects

16. How many HTTP GET request messages did the client send to retrieve the page and its embedded object? To which IP addresses were these GET requests sent?
17. Can you tell whether the client downloaded the two files (the HTML and the image) serially, or whether they were downloaded in parallel? Explain.

5.5 HTTP Authentication

18. What is the server's response status code and phrase in response to the initial HTTP GET message from the client for the protected page?
19. When the client sends the HTTP GET message for the second time, what new field is included in the request headers? What is the credentials string, and what does it decode to?

---

6. What to Hand In

Submit a lab report containing:

1. A screenshot of your CML topology showing the connected Firefox Client and Nginx Server nodes.
2. A screenshot of your Firefox browser window showing the successful display of one of the test pages alongside the developer tools Network tab.
3. The downloaded `.pcap` capture file.
4. A screenshot of your Wireshark window showing the `Authorization: Basic` header expanded in the packet details pane.
5. Your written answers to the 19 questions in Section 5.